EPSR-test-ex44 readpdb: CONECT merging and dihedral groups

Program(s): readpdb | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `readpdb`, which converts a Protein Data Bank file [1] into an EPSR `.ato` file [2]: molecules from residue numbers, merged by `CONECT` records (non-contiguous and contiguous), bonds from van der Waals radii, bond-angle constraints, a dummy site for an unknown element and `DIH` rotational groups with their dihedral angles.

Code exercised

Fortran (original, fortran_src/):

- `readpdb.f`: `run_readpdb`, `readpdb` (CRYST1/ATOM/CONECT records, molecules from residues, bonds from van der Waals radii, angle constraints, DIH groups), `upcase`
- `ato_manipulation_rev4.f`: `ctype`, `ftype`, `centreofmass`, `get_rotgrp`, `dihedral`
- `auxiliary_routines.f`: `convertalphabetagamma`; `ato_read_and_save.f`: `moloutc`

Rust (port, rust/epsr-core/src/):

- `tools/readpdb.rs`: `run`, `readpdb`, `conect` (molecule merging), `make_dihedrals`, `read_fixed`, `upcase`
- `tools/readxyz.rs`: `set_type_parameters`; `ato.rs`: `State::ctype`, `State::ftype`
- `tools/groupc_common.rs`: `get_rotgrp`, `dihedral`, `convertalphabetagamma`, `setup_lattice`, `centreofmass`, `moloutc` (with `DIH` groups)

Physical system

A small molecular mixture in a cubic 30 Å cell (CRYST1), 57 ATOM records in 15 residues, giving 13 molecules: two dimethyl sulphoxide molecules (coordinates from EPSR-test; one split into a heavy-atom residue and a separate H residue that CONECT merges back), nine waters, one TIP4P-like water whose `MW` site becomes a dummy `Q` atom, and one hand-made methanol split over two residues. Bonds are made where the distance is below 0.7 × the sum of the van der Waals radii (`vanderWaalsRadii.txt` from EPSR-test).

Inputs

`mixture.pdb` generated from the EPSR-test DMSO and water coordinates by a helper script (a one-off script that was not kept; the generated file is the input); atom serials start at 101 to test the serial look-up; a short `CONECT 9999` record is ignored. stdin: factor `0.7`, dihedrals `y`.

How the accuracy was verified

`commands.txt` (1 program call, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. `readxyz`/`readpdb` crash (SIGSEGV) as compiled from the original source, so the reference build adds an explicit INTERFACE block for `ctype` (`rust/tools/build_patched_readxyz_readpdb.sh`); the port behaves like the patched program. Result of the final regression run of the complete port: every file identical in 1 pass.
Output compared: `mixture.ato` (13 molecules, non-cubic header, DIH groups and angles). Matching required the port to print NaN as gfortran does, unsigned (`fortran/format.rs`). Fortran reference: the original `readxyz`/`readpdb` binaries crash (SIGSEGV) at the first call of `ctype`, which has optional arguments but is called with 11 of its 13 arguments without an explicit interface (gfortran passes the hidden character lengths where `radmaxsph`/`pseudofacsph` are expected). The reference was therefore produced with an explicit `INTERFACE` block for `ctype` added to the program (no other change; absent arguments mean `radmaxsphere` = `0.5*sigcor`, `pseudofacmol` = `1`), built by `rust/tools/build_patched_readxyz_readpdb.sh`; the comparison is against this patched build, not the original binary.

Notes

Fortran quirk reproduced: `dihedral(im,...)` is called with absolute atom numbers although it adds `naf(im)-1`, so only molecule 1 gets correct DIH angles; later molecules use unrelated atoms and the methanol values are NaN.

References

[1] wwPDB, "Atomic Coordinate Entry Format Description", version 3.3 (2012), https://www.wwpdb.org/documentation/file-format.
[2] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[3] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.